

Translated

From: e-maxx.ru

Lyndon factorization

Lyndon factorization

First let us define the notion of the Lyndon factorization.

A string is called **simple** (or a Lyndon word), if it is strictly **smaller than** any of its own nontrivial **suffixes**. Examples of simple strings are: a , b , ab , aab , abb , $ababb$, $abcd$. It can be shown that a string is simple, if and only if it is strictly **smaller than** all its nontrivial **cyclic shifts**.

Next, let there be a given string s . The **Lyndon factorization** of the string s is a factorization $s = w_1 w_2 \dots w_k$, where all strings w_i are simple, and they are in non-increasing order $w_1 \geq w_2 \geq \dots \geq w_k$.

It can be shown, that for any string such a factorization exists and that it is unique.

Duval algorithm

The Duval algorithm constructs the Lyndon factorization in $O(n)$ time using $O(1)$ additional memory.

First let us introduce another notion: a string t is called **pre-simple**, if it has the form $t = w \overline{w}$, where w is a simple string and $\overline{w}$ is a prefix of w (possibly empty). A simple string is also pre-simple.

The Duval algorithm is greedy. At any point during its execution, the string s will actually be divided into three strings $s = s_1 s_2 s_3$, where the Lyndon factorization for s_1 is already found and finalized, the string s_2 is pre-simple (and we know the length of the simple string in it), and s_3 is completely untouched. In each iteration the Duval algorithm takes the first character of the string s_3 and tries to append it to the string s_2 . If s_2 is no longer pre-simple, then the Lyndon factorization for some part of s_2 becomes known, and this part goes to s_1 .

Let's describe the algorithm in more detail. The pointer i will always point to the beginning of the string s_2 . The outer loop will be executed as long as $i < n$. Inside the loop we use two additional pointers, j which points to the beginning of s_3 , and k which points to the